

Basic Load Balancer Concepts

Introduction

A **Load Balancer (LB)** is one of the most important components in scalable distributed systems.

Whenever a service is horizontally scaled (multiple servers), traffic must be distributed intelligently.

That job is done by the load balancer.

Definition

A Load Balancer is a system that:

Receives incoming requests and distributes them across multiple backend servers to improve scalability, availability, and performance.

Why Load Balancer is Needed

Without LB:

None

Users -> Single Server -> Overload -> Crash

With LB:

None

Users -> Load Balancer -> Server1

-> Server2

-> Server3

Benefits of Load Balancer

1. Horizontal Scaling

Supports many backend servers.

2. High Availability

If one server fails, route to healthy servers.

3. Better Performance

Distributes traffic evenly.

4. Zero Downtime Deployments

Remove one server, deploy, add back.

5. Security Layer

Hide internal servers.

Types of Load Balancers

1. Hardware Load Balancer

Dedicated physical appliance.

Examples:

- F5 BIG-IP
- Cisco appliances

Pros:

- High performance
- Enterprise features

Cons:

- Very expensive
- Less flexible

2. Software Load Balancer

Installed on normal servers.

Examples:

- NGINX
- HAProxy
- Envoy

Pros:

- Cheap
- Flexible
- Cloud friendly

3. LBaaS (Load Balancer as a Service)

Managed cloud service.

Examples:

- Amazon Web Services ALB / NLB
- Google Cloud Load Balancing
- Microsoft Azure Load Balancer

Pros:

- Managed
- Auto scaling
- No maintenance

Layer 4 vs Layer 7 Load Balancer

Very common interview topic.

Layer 4 Load Balancer (Transport Layer)

Works at:

- TCP
- UDP

Routes based on:

- Source IP
- Destination IP
- Port number

Does NOT inspect HTTP content.

Example

None

TCP request on port 443 -> route to server

Advantages

- Very fast
- Low overhead
- Good for raw TCP traffic

Use Cases

- Databases
- Gaming traffic
- High-speed APIs
- TCP services

Layer 7 Load Balancer (Application Layer)

Works at:

- HTTP
- HTTPS
- gRPC (HTTP/2 aware in many cases)

Can inspect:

- URL path
- Headers
- Cookies
- Hostname
- Query params

Example

None

`api.site.com/users -> User Service`

`api.site.com/orders -> Order Service`

Advantages

- Smart routing
- Authentication checks
- Rate limiting
- TLS termination
- Path-based routing
- Canary deployments

Use Cases

- Web apps
- REST APIs
- Microservices

Comparison Table

Feature	Layer 4	Layer 7
Works on	TCP/UDP	HTTP/HTTPS
Speed	Faster	Slightly slower

Reads URL/Header	No	Yes
Smart routing	No	Yes
TLS termination	Limited	Yes
Best for	Raw traffic	Web/API traffic

TLS / SSL Termination

Load balancer can decrypt HTTPS traffic.

None

Client HTTPS -> Load Balancer -> HTTP internal servers

Benefits

- Reduces encryption load on app servers
- Central certificate management

When Not to Use

If internal traffic also must stay encrypted:

Use HTTPS end-to-end.

Health Checks

LB continuously checks server health.

Example:

None

GET /health

If server unhealthy:

- Removed from rotation

Sticky Sessions

Definition

Same client routed repeatedly to same server.

Useful for stateful apps.

Example

None

User A -> Server 2

User A next request -> Server 2

Use Cases

- Shopping cart in server memory
- Legacy login session
- Stateful web apps

Problems with Sticky Sessions

- Uneven traffic
- Harder scaling
- If server dies session lost

Better Modern Alternative

Store session in:

- Redis
- DB
- JWT token

Then app servers remain stateless.

Session Replication

State copied to multiple servers.

Example:

None

Server A session -> replicate to B and C

Benefit

If A crashes, B can continue.

Cost

- More complexity
- More network overhead
- Synchronization issues

Load Balancing Algorithms

1. Round Robin

Each server gets turn.

$A \rightarrow B \rightarrow C \rightarrow A$

2. Least Connections

Send to least busy server.

3. Weighted Round Robin

Powerful server gets more traffic.

4. IP Hash

Same client IP → same server.

Used for stickiness.

5. Least Response Time

Route to fastest node.

Reverse Proxy vs Load Balancer

Reverse Proxy

Sits in front of servers and forwards requests.

Used for:

- Security
- Caching
- Compression
- TLS termination

Load Balancer

Main goal:

- Traffic distribution
- Scalability

- Availability

Important Note

Many tools like NGINX do both.

Example Architecture

None

Users



CDN



Load Balancer



App Servers



Cache / DB

In Interviews, Usually Assume LB Exists

You don't need to draw it every time unless central to discussion.

Say:

“Traffic enters through an L7 load balancer with health checks and TLS termination.”

Common Failure Scenarios

1. LB is Single Point of Failure

Use multiple LBs.

2. Wrong Health Checks

Bad node still receives traffic.

3. Sticky Session Node Crash

User loses session.

4. Sudden Traffic Spike

Use auto scaling.

Real Examples

Amazon

Uses multiple LBs globally.

Netflix

Heavy use of smart traffic routing.

Cloudflare

Global traffic balancing.

Final Summary

Load balancers are mandatory in horizontally scaled systems.

They provide:

- Scalability
- Availability
- Reliability
- Secure traffic entry
- Smooth deployments